Session 3

AADK Task 3

Compose Layouts: Text and Image

Prepared by: Pawankumar

Course : Android app Development with Gen AI

Task 1 – Text Composables Understanding

Definition of @Composable:

@Composable is a special function in Jetpack Compose that defines a piece of UI. It describes how UI elements should look and behave. These functions can be combined to build complex layouts from small reusable components.

Key Text Composables:

1. Text – Displays text on the screen. Example: Showing a title.
2. Column – Arranges elements vertically. Example: Stacking title and paragraph.
3. Row – Arranges elements horizontally. Example: Two buttons side by side.

How Column/Row affect Text:

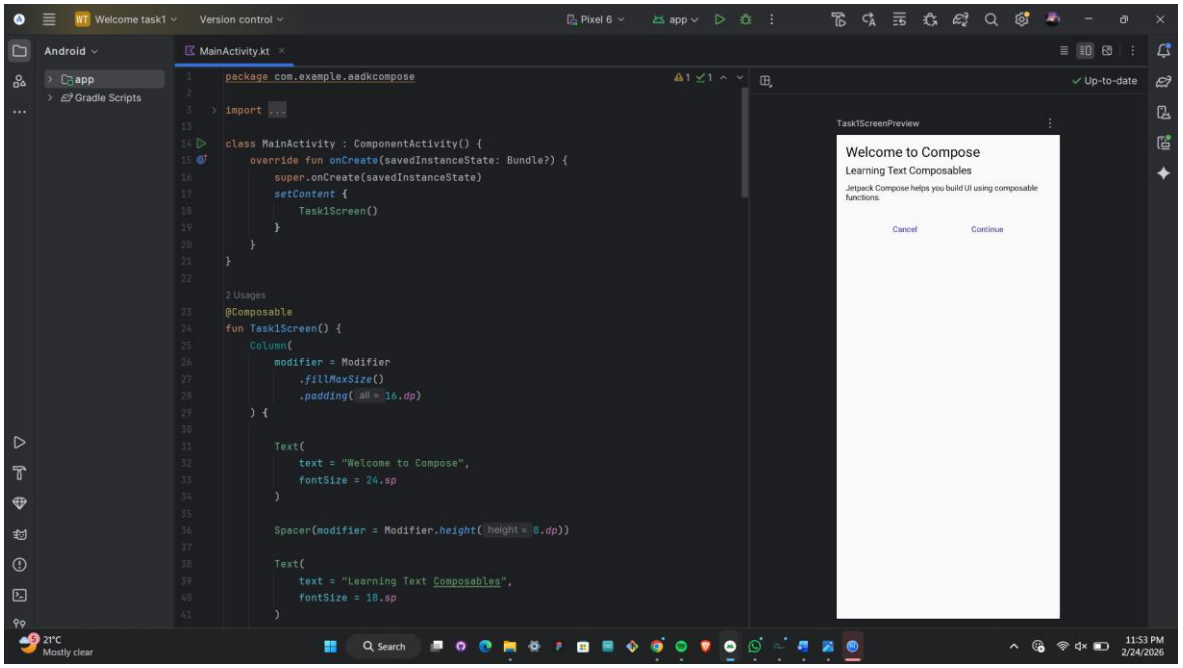
Column places text elements vertically from top to bottom. Row places them horizontally from left to right. Their arrangement changes visual hierarchy and spacing.

Task 1 Code part

```
package com.example.aadkcompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.tooling.preview.Preview

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Task1Screen()
        }
    }
}

@Composable
fun Task1Screen() {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
```

```
Text(
        text = "Welcome to Compose",
        fontSize = 24.sp
    )

    Spacer(modifier = Modifier.height(8.dp))

    Text(
        text = "Learning Text Composables",
        fontSize = 18.sp
    )

    Spacer(modifier = Modifier.height(12.dp))

    Text(
        text = "Jetpack Compose helps you build UI using composable functions.",
        fontSize = 14.sp
    )

    Spacer(modifier = Modifier.height(24.dp))

    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceEvenly
    ) {
        TextButton(onClick = { }) {
            Text("Cancel")
        }

        TextButton(onClick = { }) {
            Text("Continue")
        }
    }
  }
}

@Preview(showBackground = true)
@Composable
fun Task1ScreenPreview() {
  Task1Screen()
}
```

Task 1 output



Above image: Wireframe showing Title, Subtitle, Paragraph and Row with two TextButtons.

Task 2 – Image Inclusion & Resource Management

Step 1

Image Composable:

Image composable displays images in Compose UI using painterResource or rememberAsyncImagePainter.

Why Density Matters:

Different devices have different screen densities. If images are not optimized for density folders, they may appear blurry or too large, increasing memory usage.

Best Practices:

• Use proper naming (e.g., profile_image.png).

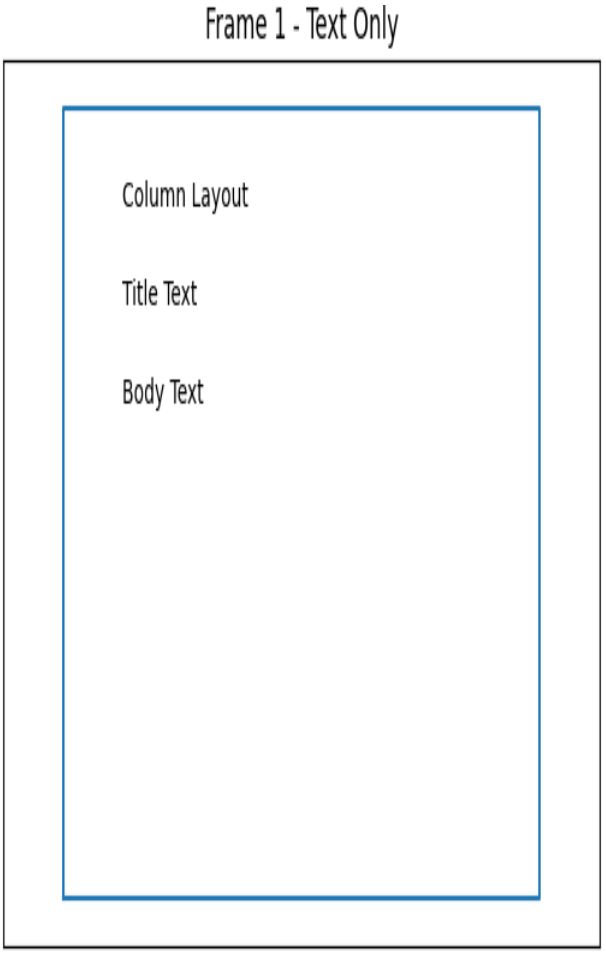
• Optimize image size before adding to drawable folder.

Image add ing process

app → res → drawable → birthday_banner.png

image name Example

birthday_banner.png

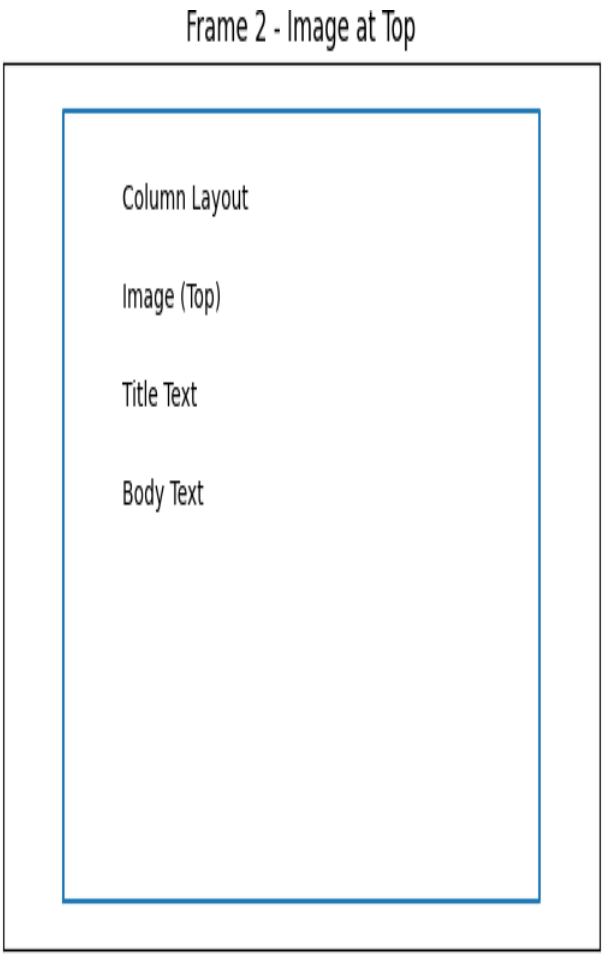


Frame 1: Screen with only text using Column.

Step : 2

```
package com.example.aadkcompose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp



class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Task2Frame2()   // Change this to Frame1 or Frame3 to test
        }
    }
}

@Composable
fun Task2ImageTop() {
Column(
modifier = Modifier
.fillMaxSize()
.padding(16.dp)
) {

Image(
painter = painterResource(id = R.drawable.birthday_banner),
contentDescription = "Birthday Banner",
modifier = Modifier
.fillMaxWidth()
.height(200.dp),
contentScale = ContentScale.Crop
)

Spacer(modifier = Modifier.height(16.dp))

Text(text = "Happy Birthday!", fontSize = 24.sp)

Spacer(modifier = Modifier.height(8.dp))
```

```
Text(text = "Wishing you success and happiness.", fontSize = 16.sp)
}

}
```

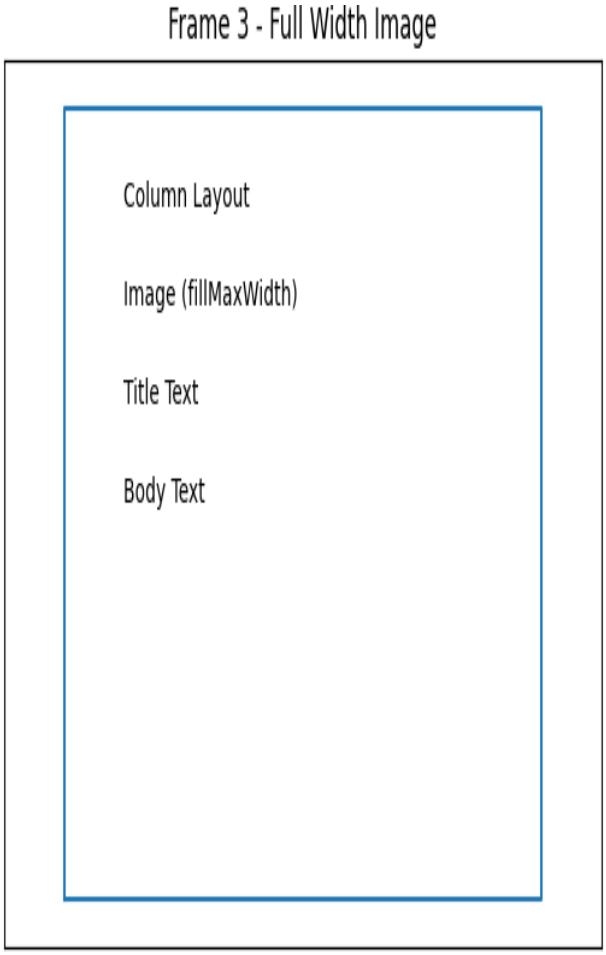


Frame 2: Image added at top of Column layout.

Step 3

```
@Composable
fun Task2FullWidthImage() {
    Column {

        Image(
            painter = painterResource(id = R.drawable.birthday_banner),
            contentDescription = "Full Width Image",
            modifier = Modifier.fillMaxWidth(),
            contentScale = ContentScale.FillWidth
        )

        Spacer(modifier = Modifier.height(16.dp))

        Column(modifier = Modifier.padding(16.dp)) {
            Text("Happy Birthday!", fontSize = 24.sp)
```

```
    Spacer(modifier = Modifier.height(8.dp))
        Text("Enjoy your special day!", fontSize = 16.sp)
    }
  }
}
```

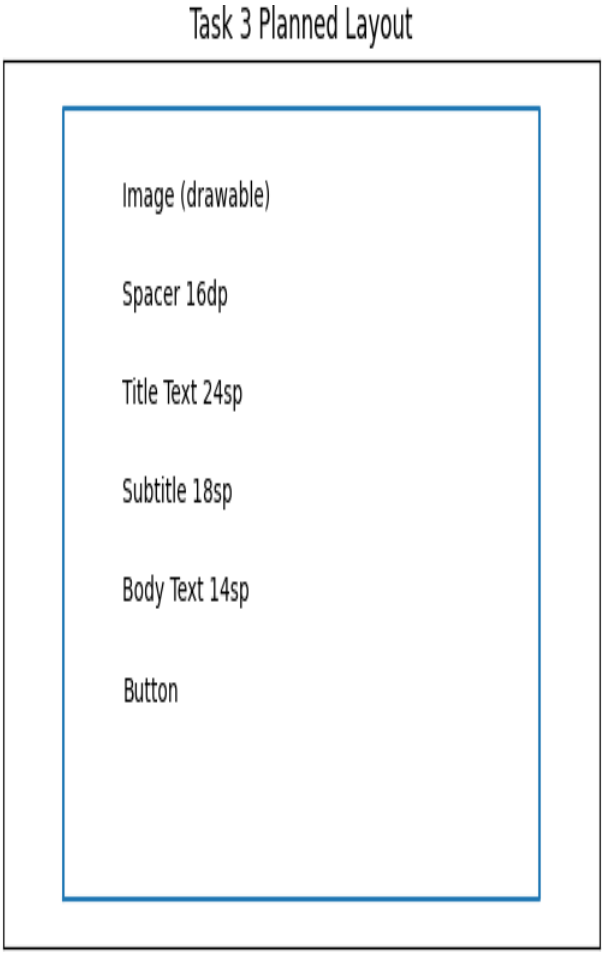


Frame 3: Image adjusted to full width using Modifier.fillMaxWidth()

Task 3 – Planning Simple Layout (Birthday Greeting Card)

Purpose:

This screen displays a birthday greeting image with personalized text and a button to send wishes.



Above image: Planned layout showing image, spacing (16dp), title (24sp), subtitle (18sp), body (14sp), and button.

Resource Plan:

Image: birthday_banner.png □ drawable-nodpi □ medium size (~200KB). Text Sizes:

Title 24sp, Subtitle 18sp, Body 14sp.

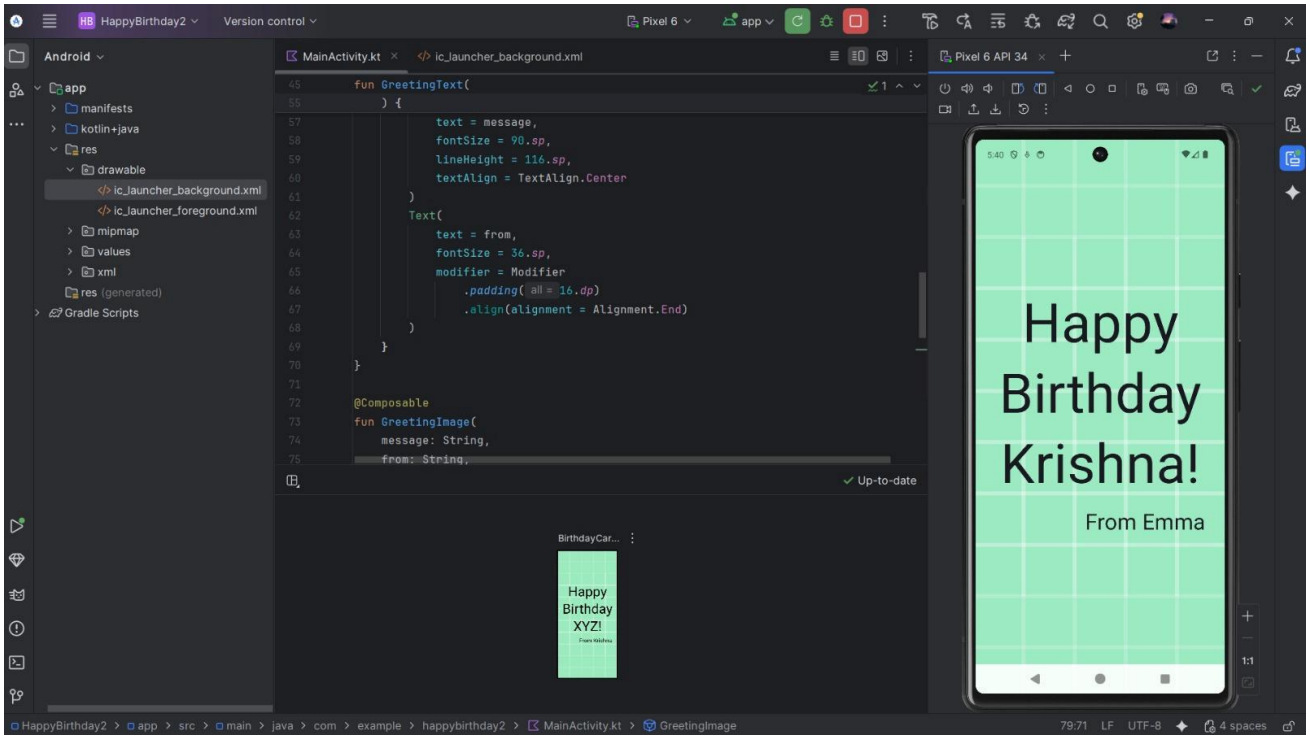
Reflection:

Planning composables and resources before coding reduces layout errors, improves UI clarity, and avoids performance issues.

Birthday Card full code

```
package com.example.myapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.myapplication.ui.theme.MyApplicationTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        setContent {
            MyApplicationTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    GreetingImage(
                        message = "Happy Birthday Krishna!",
                        from = "From Emma"
                    )
                }
            }
        }
    }
}

@Composable
fun GreetingText(
    message: String,
    from: String,
    modifier: Modifier = Modifier
) {
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
```

```
Text(
        text = message,
        fontSize = 89.sp,
          lineHeight = 78.sp,
        textAlign = TextAlign.Center
    )
    Spacer(modifier = Modifier.height(30.dp))

    Text(
        text = from,
        fontSize = 30.sp,
        modifier = Modifier.align(Alignment.End)
    )
  }
}

@Composable
fun GreetingImage(
  message: String,
  from: String,
  modifier: Modifier = Modifier
) {
  val image = painterResource(id = R.drawable.androidparty)

  Box(modifier = modifier.fillMaxSize()) {

    Image(
        painter = image,
        contentDescription = null,
        contentScale = ContentScale.Crop,
        alpha = 0.5f,
        modifier = Modifier.fillMaxSize()
    )

    GreetingText(
        message = message,
        from = from
    )
  }
}

@Preview(showBackground = true)
@Composable
fun BirthdayCardPreview() {
  MyApplicationTheme {
    GreetingImage(
        message = "Happy Birthday XYZ!",
        from = "From Krishna"
    )
  }

}
```


HappyBirthday2
Version control
Pixel 6
app
Android
app
manifests
kotlin+java
res
drawable
ic_launcher_background.xml
ic_launcher_foreground.xml
mipmap
values
xml
res (generated)
Gradle Scripts
MainActivity.kt
ic_launcher_background.xml
fun GreetingText(
) {
text = message,
fontSize = 90.sp,
lineHeight = 116.sp,
textAlign = TextAlign.Center
)
Text(
text = from,
fontSize = 36.sp,
modifier = Modifier
.padding(all = 16.dp)
.align(alignment = Alignment.End)
)
}
}
@Composable
fun GreetingImage(
message: String,
from: String,
Up-to-date
BirthdayCar...
Happy Birthday XYZ!
Pixel 6 API 34
Happy Birthday Krishna!
From Emma
HappyBirthday2 > app > src > main > java > com > example > happybirthday2 > MainActivity.kt > GreetingImage
79:71
LF
UTF-8
4 spaces

Task 4 – Accessibility & Performance Review

Element	Potential Issue	Proposed Fix	Priority
Top Image	Large raw size	Compress & resize	High
Subtitle Text	Small font	Increase to 18sp	Medium
Button	Small touch area	Minimum 48dp heig	High

Mini Action Plan:

Before coding: Optimize image size. During

coding: Add contentDescription.

After testing: Adjust font sizes for readability.

Final Reflection:

Planning accessibility and performance early ensures inclusive design, smooth performance, and better user experience across devices.